

Tesseract++: End-to-End Extraction of Multi-Floor Navigable Graph Representations from Annotated Building Floorplans

YAQOOB ANSARI, Carnegie Mellon University, USA

EDUARDO FEO FLUSHING, Carnegie Mellon University, USA

KHALED A. HARRAS, Carnegie Mellon University, USA

Indoor navigation and spatial reasoning require structured representations of building layouts. Converting architectural floorplans into machine-readable navigable graphs remains difficult because visual annotations carry no explicit topological connectivity. We present TESSERACT++, an end-to-end system that transforms annotated building floorplans into multi-floor navigable graph representations. The pipeline detects and interprets text labels to seed rooms, corridors, outdoor areas, and vertical transition elements. It then segments the drawing with flood fill, places spatial subnodes with distance-transform guidance, and detects doors with a type-aware classifier that distinguishes room-to-corridor, room-to-room, and exit doors. Graph construction uses room-family funneling, a construction with a proven guarantee that every room point reaches every room door while the number of intra-room edges stays linear in the subnode count. For multi-floor buildings, TESSERACT++ infers vertical correspondences automatically. A registration stage aligns adjacent floor drawings using wall evidence and a drawing-scale prior, and a globally optimal assignment with a rejection option pairs stairs and elevators across floors while leaving terminating shafts unmatched. Hand-written mappings remain available as an optional override, and floor indices are read from a filename convention. We evaluate the system with degradation studies, ablations, an objective route-fidelity reference based on free-space geodesics, and a quantitative baseline comparison. Graph routes stay within 4.7% of the shortest walkable paths on the primary evaluation floor, and pruning removes 59% of nodes and 82% of edges with no measured route inflation.

CCS Concepts: • **Information systems** → **Geographic information systems**; • **Computing methodologies** → **Computer vision**; • **Theory of computation** → *Graph algorithms analysis*.

Additional Key Words and Phrases: floorplan parsing, indoor navigation, spatial graphs, semantic mapping, door detection, multi-floor connectivity, building information extraction, graph construction

ACM Reference Format:

Yaqoob Ansari, Eduardo Feo Flushing, and Khaled A. Harras. 2026. Tesseract++: End-to-End Extraction of Multi-Floor Navigable Graph Representations from Annotated Building Floorplans. 1, 1 (August 2026), 23 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 INTRODUCTION

Indoor navigation depends on structured representations of architectural layouts. Robotic path planning and building accessibility analysis share the same dependency. Outdoor navigation benefits from mature geographic information systems and standardized road network graphs. Indoor environments offer no such standard. Buildings exhibit complex

Authors' addresses: Yaqoob Ansari, Carnegie Mellon University, Pittsburgh, PA, USA, yansari@andrew.cmu.edu; Eduardo Feo Flushing, Carnegie Mellon University, Pittsburgh, PA, USA, efeo@andrew.cmu.edu; Khaled A. Harras, Carnegie Mellon University, Pittsburgh, PA, USA, kharras@andrew.cmu.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Association for Computing Machinery.

Manuscript submitted to ACM

Manuscript submitted to ACM

multi-room topologies, vertical connectivity through stairs and elevators, and semantic distinctions between navigable corridors, restricted areas, and transition zones. Converting architectural floorplans, the primary documentation of building layouts, into machine-readable navigable graphs therefore remains a significant technical challenge.

Legacy floorplans dominate the available documentation. Existing approaches to indoor map generation often require manual annotation, assume access to Building Information Models (BIM), or rely on sensor-based simultaneous localization and mapping (SLAM). Vast repositories of institutional, commercial, and residential floorplans instead exist as raster images with minimal semantic markup. These drawings contain text annotations identifying rooms and spaces, standard symbols for doors, and dark strokes for walls. They lack the structured connectivity information that navigation applications require.

This manuscript substantially extends our prior conference publication [3], which introduced the core single-floor parsing pipeline. The extension centers on two contributions. First, a multi-floor graph construction module that infers vertical correspondences between floors automatically, registers adjacent floor drawings into a common frame, and merges per-floor graphs into a unified building representation. Second, a room-family representation with distance-transform subnodes and a funneling construction whose connectivity guarantee we state and prove. The remaining additions support these two contributions:

- Automatic inter-floor transition matching, formulated as wall-evidence registration followed by optimal assignment with a rejection option, with hand-written mappings retained as an override.
- A proven funneling guarantee, complete intra-room connectivity at linear intra-room edge cost, validated by a three-way ablation against dense and single-node room encodings.
- A deepened evaluation of type-aware door connectivity, including degradation studies that quantify how detector errors propagate to graph connectivity.
- An expanded experimental methodology covering text accuracy, corridor density, pruning ablations, an objective route-fidelity reference, downstream navigation tasks, and a quantitative baseline.

TESSERACT++ operates as a modular pipeline. Given raster floorplan images with text annotations and standard architectural conventions, it produces typed, attributed graphs in which nodes represent spatial regions and edges encode traversability with geometric distance weights. Text detection and semantic interpretation identify labeled regions. Flood-fill segmentation with subnode placement encodes spatial extent. Door detection and classification establish connectivity. Graph construction with pruning produces well-conditioned outputs for pathfinding algorithms.

A hierarchical room representation underlies the graph construction. Rooms in architectural floorplans may span large areas with complex shapes, yet navigation requires fine-grained spatial encoding. Each semantic room therefore contains a designated main node and multiple subnodes sampled on a hexagonal grid within the room boundary. Room-family funneling connects each family through a shortest-path tree rooted at the main node, plus a single shortcut edge from each door to its closest family node. This construction guarantees that every subnode reaches every room door while spending only linearly many intra-room edges. Subnodes adjacent to a door exit directly rather than detouring through the room center.

Multi-floor construction no longer depends on hand-written correspondences. The system registers adjacent floor drawings using wall-mask evidence and a drawing-scale prior bootstrapped from detected door widths. It then solves a global assignment problem over the detected stairs and elevators, with an explicit rejection option so that shafts without a counterpart remain unmatched. Validation rules that previously checked a human’s input now verify the machine’s

answer. Two elements remain metadata-assisted by design. Floor indices are read from a filename convention, and a hand-written mapping file can override the inferred correspondence.

The remainder of this paper is organized as follows. Section 2 surveys related work. Section 3 defines the problem and output schema. Section 4 presents the system architecture. Sections 5–9 detail the pipeline components. Section 10 presents the experimental evaluation. Section 11 discusses limitations. Section 12 concludes.

2 RELATED WORK

Floorplan Recognition and Parsing. Learned floorplan parsing has advanced rapidly. Raster-to-vector conversion recovers junction and wall structure from rasterized drawings [19], CubiCasa5k provides a large annotated corpus with a multi-task parsing network [15], and room-boundary-aware networks improve wall and opening recovery [10]. FloorNet reconstructs floorplans from RGBD scans rather than drawings [18]. These systems target geometric reconstruction. They recover walls, rooms, and openings, but they do not produce navigable graphs with typed nodes, traversability edges, or vertical connectivity. TESSERACT++ consumes the same class of raster inputs and outputs a routing-ready representation directly.

Indoor Mapping and Localization. Sensor-driven pipelines build maps by traversing the building. SLAM systems produce metric maps from robot or handheld sensing [8], and WiFi fingerprinting supports localization once a signal survey exists [5]. Both require physical access, instrumentation, and survey effort per building [2]. Floorplan-based extraction sidesteps deployment cost entirely because the drawing already exists, which makes it attractive for large legacy portfolios.

Graph-Based Spatial Representations. Navigation systems consume topological graphs rather than raster maps. IndoorGML standardizes indoor spaces as cellular graphs with connectivity relations [16], and classic shortest-path algorithms operate directly on weighted graphs [9, 13]. Wayfinding research further motivates typed nodes because route semantics matter to users [20], and accessibility analysis requires explicit transition elements [1]. Our output schema follows this line. Nodes are typed, edges carry geometric weights, and stairs and elevators are first-class objects.

Scene Text Detection and Recognition. The Character Region Awareness for Text (CRAFT) detector predicts character-level region and affinity heatmaps and achieves strong accuracy on scene text benchmarks [4]. EAST offers an efficient single-stage alternative [24]. For recognition, convolutional recurrent architectures with Connectionist Temporal Classification (CTC) decoding remain the standard for unconstrained strings [11, 22]. We adopt CRAFT for detection and a CTC-based recognizer for interpretation because floorplan labels are short, sparse, and often rotated.

Architectural Element Detection. Door symbols are small, stylized, and easily confused with other arcs. We build on Faster R-CNN [21] with a ResNet backbone and feature pyramid [14, 17], fine-tuned on floorplan door symbols. Generic detectors transfer poorly to drawings without such fine-tuning because the symbol distribution differs sharply from natural images.

Multi-Floor Building Representations. BIM and IFC models encode complete vertical structure [7, 23], but they exist mainly for recent construction and demand heavyweight tooling. Lightweight navigation graphs derived from per-floor drawings must instead recover vertical connectivity after the fact. To our knowledge, automatic inference of stair and elevator correspondences from raster floor drawings alone has not been addressed, and this gap motivates the matching module of Section 9.

3 PROBLEM SETTING AND OUTPUT SCHEMA

3.1 Input Assumptions

The underlying representation is raster image space, and the pixel is the atomic unit of every pipeline stage. TESSER-
ACT++ accepts floorplan images satisfying the following assumptions:

- (1) **Text annotations.** Rooms and spaces carry text labels, and semantic classes are seeded exclusively by these labels rather than by fill colors. Table 1 lists the recognized vocabulary.
- (2) **Architectural symbols.** Doors follow standard drafting conventions with arc symbols indicating swing direction.
- (3) **Wall encoding.** Walls appear as dark strokes recoverable by intensity thresholding.
- (4) **Floor naming.** For multi-floor processing, filenames encode floor identity, with FF, SF, and TF denoting the first, second, and third floors, B1 and B2 denoting basements, and numeric prefixes mapping directly.

Table 1. Label vocabulary. Interpretation is case-insensitive with single-edit fuzzy matching, and every semantic class must be present for door classification to proceed.

Label on drawing	Interpreted as
any numeric token (e.g., 1001)	room, with the number as its name
hall, hallway, corridor, passage	corridor region
na	outside or non-navigable region
stairs, stair, staircase	stairs transition node
elev, elevator, lift	elevator transition node

These assumptions define an explicit operating envelope. The system targets institutional and commercial drawings that follow standard drafting conventions and carry the label vocabulary above. It does not target hand-drawn sketches, sparsely annotated plans, or drawings with non-standard door symbols. A drawing that lacks corridor, outside, or transition labels fails the semantic-class requirement and is rejected rather than silently mis-segmented, and Section 10.2 demonstrates this failure mode on an unlabeled variant of an evaluation floor.

3.2 Output Graph Schema

The output is a typed, attributed graph $G = (V, E)$ with node set $V = V_{\text{room}} \cup V_{\text{corridor}} \cup V_{\text{outside}} \cup V_{\text{door}} \cup V_{\text{transition}}$.

Each node $v \in V$ carries the following attributes:

- **id**, a unique string identifier
- **type**, one of room, corridor, outside, door, or transition
- **position**, the (x, y) pixel coordinates
- **floor**, an integer floor identifier
- **pixels**, the region’s pixel set for area nodes

Each edge $e = (u, v) \in E$ carries the following attributes:

- **weight**, the Euclidean distance between node positions
- **distance**, the geometric distance in pixels
- **edge_type**, which distinguishes intra-floor from inter-floor edges in merged graphs

The graph is undirected. An exit door is defined as a door whose two incident regions are an interior space and an outdoor region. One-way restrictions and emergency-only egress are not modeled, and Section 11 lists directional edge attributes as the natural extension.

For buildings with multiple floors, the merged graph uses floor-prefixed node identifiers such as 1_stairs_1 to ensure uniqueness, and it preserves the original identifier in a node attribute.

3.3 Downstream Applications

The output graph supports the following tasks:

- **Pathfinding** with Dijkstra or A* over distance weights
- **Accessibility analysis**, including step-free routing over typed transitions
- **Egress analysis** through exit doors
- **Spatial queries** over room adjacency and floor connectivity
- **Integration** through JSON export compatible with GIS tools

Section 10.9 validates the first three tasks directly instead of leaving the application claims implicit.

4 SYSTEM OVERVIEW

Figure 1 illustrates the TESSERACT++ pipeline. Processing proceeds through six stages:

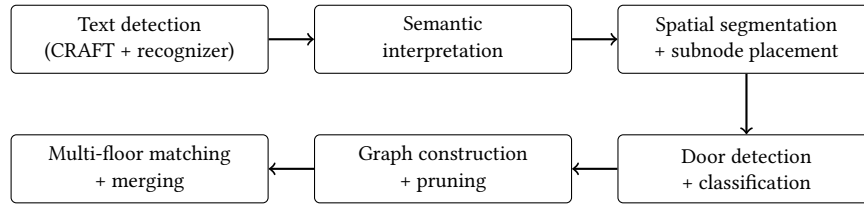


Fig. 1. Pipeline architecture. Each single-floor stage feeds the next, and the multi-floor stage consumes the per-floor graphs together with the floor drawings.

- (1) **Text Detection and Recognition** (Section 5). CRAFT-based detection followed by CTC-based recognition extracts text annotations.
- (2) **Semantic Interpretation** (Section 5). Detected text seeds room labels, corridor markers, outdoor indicators, and transition elements.
- (3) **Spatial Segmentation** (Section 6). Flood fill segments the drawing, and distance-transform-guided sampling places subnodes.
- (4) **Door Detection and Classification** (Section 7). Faster R-CNN detects door symbols, and refinement plus classification distinguishes door types.
- (5) **Graph Construction and Pruning** (Section 8). Nodes and edges are assembled and pruned while preserving reachability.
- (6) **Multi-Floor Connectivity** (Section 9). Floor graphs are merged, and transitions are matched across floors automatically.

Timing Characteristics. Single-floor processing takes about 52 seconds for a 1946×1574 image on a standard CPU. Door detection dominates at about 21 seconds, followed by flood-fill segmentation at about 9 seconds. Multi-floor matching and merging add under two seconds. Section 10.10 gives the full breakdown.

5 TEXT DETECTION AND SEMANTIC INTERPRETATION

5.1 Text Region Detection

Text regions are located with the CRAFT detector [4]. CRAFT predicts a region score indicating character centers and an affinity score encoding character adjacency, and connected component analysis on the combined heatmap yields word-level boxes. The detector outputs quadrilateral boxes, which accommodates the rotated text common in architectural drawings.

5.2 Text Recognition

Detected regions are recognized by a sequence model. The recognizer combines a Visual Geometry Group (VGG) convolutional feature extractor, a bidirectional Long Short-Term Memory (BiLSTM) sequence encoder, and a Connectionist Temporal Classification (CTC) decoder [11, 22]. Each cropped region is normalized before decoding.

Recognition output is normalized by semantic context rather than by generic spell correction. Tokens are matched against the closed label vocabulary of Table 1 with case folding and single-edit fuzzy tolerance. An alphabetic token of three or more characters matches a vocabulary word within Levenshtein distance one, which absorbs single-character OCR confusions such as nna for na. A token within distance one of words from two different classes is rejected as ambiguous. Numeric tokens never fuzzy-match because they are room identifiers, and alphabetic look-alikes inside otherwise numeric tokens are instead mapped to digits, which resolves confusions such as the letter O for the digit zero.

5.3 Semantic Interpretation

The interpretation module classifies recognized strings into the classes of Table 1. Numeric tokens become room identifiers, and the text position seeds the room's main node. Corridor and outside keywords seed region segmentation. Transition keywords produce typed stairs and elevator nodes, which later anchor inter-floor matching. The module outputs a mapping from text regions to semantic types that seeds all subsequent stages.

6 SPATIAL SEGMENTATION AND REGION ENCODING

6.1 Wall Mask Extraction

Walls are extracted by intensity thresholding. Near-black pixels form the wall mask, and morphological dilation expands it to separate adjacent regions. Text strokes also fall below the threshold and therefore enter the wall mask, and we measured them at roughly 9% of wall ink on the primary evaluation floor. This contamination is harmless during segmentation because flood fill treats glyphs as small obstacles and the dilation buffer dominates. The registration module of Section 9, where wall masks serve as alignment evidence, removes glyphs explicitly by connected-component filtering.

6.2 Flood Fill for Room Segmentation

Rooms are segmented by flood fill from their labels. The algorithm identifies a seed pixel near the text position that lies in unfilled space away from walls, performs bounded flood fill that respects wall boundaries, and records the filled pixels as the room’s region. We fill at pixel granularity rather than polygonizing wall boundaries because legacy drawings frequently contain hairline gaps, decorative strokes, and symbol overlaps that make closed-polygon extraction brittle, whereas flood fill degrades gracefully. Vectorized boundaries remain attractive future work.

6.3 Subnode Placement

Large rooms need multiple nodes to encode spatial extent. We employ distance-transform-guided sampling:

- (1) Compute the distance transform of the room mask [6].
- (2) Generate a hexagonal grid over the room with 60-pixel spacing.
- (3) Retain grid points inside the mask whose distance to the boundary exceeds a 10-pixel padding threshold.
- (4) Place the main node at the room label and the retained grid points as subnodes.

Using the label position for the main node is empirically justified rather than assumed. Across 172 rooms on nine processed floor sections, the offset between the label position and the geometric centroid of the flood-filled region has a median of 0.24 room-equivalent radii, and 92% of labels fall inside the room’s inscribed circle. Section 10.2 reports the full distribution.

6.4 Corridor and Outdoor Sampling

Corridors and outdoor areas receive grid sampling rather than hexagonal packing. The difference from rooms is the seeding. Rooms are compact labeled regions that flood fill can grow from a single seed, whereas corridors are elongated regions in which a uniform grid over the corridor mask, with wall-buffered filtering, yields evenly spaced nodes and eight-connected lattice edges. Section 10.5 quantifies the effect of the grid spacing on graph size, connectivity, and route quality.

7 DOOR DETECTION PIPELINE

7.1 Door Symbol Detection

Door symbols are detected with Faster R-CNN [21] using a ResNet-50 backbone with a feature pyramid network [14, 17], fine-tuned on architectural door symbols. Raw detections undergo non-maximum suppression and geometric validation. Retained doors must lie near wall boundaries, have plausible aspect ratios, and avoid excessive overlap with other detections.

7.2 Door Classification

Detected doors are classified by the regions they connect. Room-to-corridor doors are the primary access points. Room-to-room doors connect suites directly. Corridor-to-corridor doors join corridor sections through fire doors and dividers. Exit doors connect an interior space to an outdoor region and anchor egress analysis. Classification uses spatial proximity, and each door takes the type implied by its two nearest non-door nodes.

7.3 Room-Family Funneling

Room-family funneling provides complete intra-room connectivity at linear edge cost. A room encoded by many subnodes must connect to its doors without the quadratic blow-up of dense subnode-to-door wiring, and realized paths must stay short. Algorithm 1 states the construction as implemented.

Algorithm 1 Room-Family Funneling

Require: Room r with main node m , subnodes S , associated doors D_r , lattice spacing σ

Ensure: Every node of $F = \{m\} \cup S$ reaches every door in D_r , with $O(|S|)$ intra-room edges

- 1: Build a transient proximity lattice L on F that connects $u, v \in F$ iff $\|u - v\| \leq 1.25\sigma$
 - 2: For each door $d \in D_r$ keep the edge (c_d, d) to the closest family node c_d , and for room-to-corridor and exit doors additionally keep (m, d)
 - 3: **for** each node $u \in F$ **do**
 - 4: Mark the edges of the shortest path $u \rightsquigarrow a$ in L , where the anchor a is m for room-to-corridor and exit doors and c_d otherwise
 - 5: **end for**
 - 6: Delete all unmarked lattice edges, so the marked edges form a tree on F rooted at a
-

PROPOSITION 7.1 (SPARSE COMPLETE INTRA-ROOM CONNECTIVITY). *Let room r have family $F = \{m\} \cup S$ whose proximity lattice L is connected, and doors D_r , each retaining at least one edge into F . After funneling, (i) every node of F reaches every door in D_r , and (ii) the number of intra-room edges is at most $|S|$, versus $\Theta(|S| \cdot |D_r|)$ for dense subnode-to-door wiring.*

PROOF. The marked edge set is a union of shortest paths in L from every node of F to a common anchor $a \in F$. A union of root-directed shortest paths is a tree spanning F , hence connected with exactly $|F| - 1 = |S|$ edges, and fewer if global pruning later removes leaf subnodes. Every door $d \in D_r$ retains the edge (c_d, d) with $c_d \in F$, so any $u \in F$ reaches d via the tree path $u \rightsquigarrow c_d$ followed by that edge. $\square$

We state the guarantee precisely because it differs from the conference phrasing. Paths are not forced through the main node. A subnode adjacent to a door exits directly through the closest-node edge (c_d, d) , and on our primary evaluation floor 69 of 186 subnode-to-door shortest paths bypass the main node. This behavior is intentional. The property downstream routing needs is completeness at linear cost, and the shortcut edge strictly improves realized path lengths. Section 10.4 quantifies the trade-off with a three-way ablation.

Construction and Pruning Complexity. Funneling runs one Dijkstra per family node on the lattice L , which costs $O(|F|^2 \log |F|)$ per room with small constants because hexagonal packing bounds the lattice degree by six. Global pruning then computes bidirectional Dijkstra shortest paths for all $\binom{R}{2}$ main-room pairs, $|X| \cdot R$ exit-door pairs, and $|T| \cdot R$ transition pairs, where R , $|X|$, and $|T|$ count main rooms, exit doors, and transitions. On the primary evaluation floor with $R = 34$, $|X| = 2$, $|T| = 3$, and 2,139 pre-pruning nodes, this amounts to 963 shortest-path computations completing in 2.0 seconds single-threaded. Pruning is therefore not the bottleneck, since door detection dominates at about 21 seconds.

7.4 Transition Node Integration

Transition elements require dedicated wiring into the corridor network. Transition nodes are identified during semantic interpretation. For each transition, the system traces from the shaft toward the nearest corridor region, places a

synthetic entry door at the first wall crossing, and connects the transition through that entry door to the corridor lattice. A post-pruning fallback guarantees that every transition retains a corridor connection. For multi-floor processing these nodes serve as the anchors of inter-floor matching.

8 GRAPH CONSTRUCTION AND PRUNING

8.1 BuildingGraph Data Structure

The BuildingGraph class encapsulates graph state over a NetworkX representation [12]. It exposes typed node insertion, edge creation with automatic Euclidean weights, JSON export with sanitized attributes, and visualization overlaid on the source drawing.

8.2 Edge Creation

Edges arise from four mechanisms:

- **Intra-room edges**, the funneling tree of Algorithm 1 plus one closest-node edge per door and a main-node edge for room-to-corridor and exit doors
- **Corridor lattice edges**, eight-connected links between adjacent corridor grid nodes
- **Door-mediated edges**, connecting each door to the regions its classification implies
- **Transition edges**, wiring stairs and elevators through entry doors into the corridor network

8.3 Graph Pruning

Dense sampling makes the initial graph highly redundant. Pruning computes a keep-set as the union of four families of shortest paths, namely intra-family paths, all main-room pairs, the best main-room path to each exit door, and all main-room-to-transition paths. Edges and nodes outside the keep-set are removed, and transition nodes are never pruned. [Transitions can reach degree zero before this protection exactly when no room-to-transition path exists, for instance when a missed door seals a shaft, and the keep-term ablation of Section 10.4 shows the protection is load-bearing.](#)

Pruning reduces the primary evaluation floor from 2,139 to 878 nodes and from 6,262 to 1,118 edges, a 59% node and 82% edge reduction. Section 10.4 shows that this reduction costs nothing in route quality.

9 MULTI-FLOOR CONNECTIVITY

9.1 Floor Detection and Graph Merging

Floor identity comes from a filename convention. The tokens FF, SF, and TF map to floors one to three, B1 and B2 map to basements, and numeric prefixes map directly. Merging then prefixes every node identifier with its floor number, preserves the original identifier and floor as attributes, and copies intra-floor edges with updated references.

9.2 Automatic Transition Matching

The central multi-floor question is which stairs and elevators on adjacent floors are the same physical shaft. The conference system delegated this question to a hand-written mapping file. The revised system answers it automatically in two stages, registration and assignment.

Registration. Registration estimates a similarity transform that maps one floor’s pixel frame onto the other. Candidate transforms come from four generators, namely pairs of same-kind transition correspondences with mirrored

variants, per-image dimension normalization anchored on single candidate pairs, phase correlation of the wall masks at four drawing orientations, and a drawing-scale prior obtained from detected door widths, since interior door leaves follow a standardized width of about 0.9 meters. Candidates outside a plausibility gate on scale and rotation are discarded. Remaining candidates are ranked by wall evidence, a symmetric chamfer distance between glyph-filtered wall masks that is gated on mutual coverage, with a small bonus per one-to-one aligned transition pair. The winning transform is refined by least squares on its aligned pairs and by iterative closest point on wall ink.

Assignment with rejection. Matching is a global assignment problem rather than a chain of local decisions. Each candidate pair of transitions receives a cost with three terms, the distance between the registered positions, a heavy penalty for pairing stairs with elevators, and a light term comparing graph-context features such as corridor density and distance to exits. The Hungarian solver runs on this cost matrix augmented with a rejection slot per shaft, and a pair is accepted only if its cost stays below the rejection price. Shafts whose best pairing exceeds that price remain unmatched, which is the desired behavior for a stairwell that terminates or an elevator that serves only some floors.

Provenance and verification. The inferred correspondence is written as a standard mapping file with full provenance, including the registration transform, wall-agreement score, coverage, and per-match cost and confidence. Explicit warnings flag weak wall agreement, mirrored registration, and correspondences supported by fewer than two aligned pairs. The validation rules of the conference system, namely image existence, floor adjacency, one-to-one mapping, and node typing, now verify the machine’s answer instead of a human’s input. A hand-written mapping file remains available as an override and takes precedence when provided.

9.3 Inter-Floor Edge Weights

Inter-floor edges carry modeled vertical travel instead of a nominal cost. The weight is the product of a travel factor, the story height, and the drawing scale in pixels per meter. The scale is bootstrapped from the median short side of detected door boxes under the standardized 0.9-meter leaf width, the story height defaults to 4 meters, and the travel factor is 2.2 for stairs and 1.0 for elevators. The estimated scales are independently consistent across floors of our evaluation building, 31.1 versus 32.2 pixels per meter, which supports the bootstrap. When no scale can be estimated the system falls back to a unit weight and emits a warning.

9.4 Connectivity Verification

Post-merge verification is exhaustive rather than sampled. The system reports the connected-component structure of the merged graph, pairwise room completeness, and per-floor integrity, using the metric layer of Section 10.4.

10 EXPERIMENTAL EVALUATION

10.1 Dataset and Setup

The evaluation uses annotated sections of a university building together with ten stylistically distinct smaller plans. Figure 2 shows two representative drawings, and Table 2 summarizes the corpus. [Results for the additional multi-floor site and the consolidated three-floor dataset will be incorporated in this subsection once the extended runs complete.] All experiments run single-threaded on a standard CPU, and every experiment in this section is reproducible from a released evaluation suite.

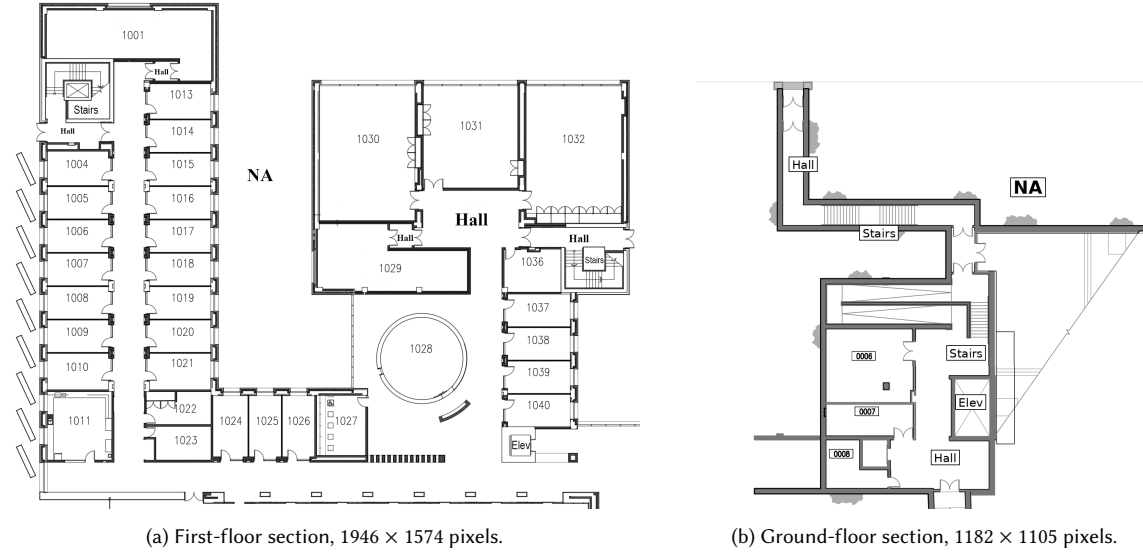


Fig. 2. Representative evaluation drawings. Rooms carry numeric labels, corridors carry hall labels, outdoor regions carry na labels, and stairs and elevators are labeled at their shafts. Walls are dark strokes and doors use standard arc symbols.

Table 2. Evaluation corpus. Room, exit, and transition counts refer to the post-pruning graphs.

Plan	Size (px)	Rooms	Exits	Transitions
First-floor section A	1946 × 1574	34	2	3
First-floor section B	1596 × 1120	35	10	0
First-floor section C	1122 × 1126	28	3	0
First-floor section D	1712 × 1082	20	2	0
Ground-floor section	1182 × 1105	3	0	3
Ten smaller plans	various	2–6 each	0–6 each	0

10.2 Text and Semantics

Label detection is measured against a hand-enumerated ground truth of every label visible on the drawing. On first-floor section A the detector finds 43 of 43 labels, and recognition is exact for 41 of 43. Both failures truncate the leading digit of a room number, for example 1011 becomes 101. The consequences differ by failure class. A misrecognized numeric label still yields a valid room node with a wrong name, so connectivity is unaffected. A label missed entirely leaves its region without a semantic seed, and the region is not instantiated. An unlabeled variant of the same drawing demonstrates the extreme case by producing zero transition nodes and failing the five-class requirement of door classification.

Label placement supports the main-node choice. Across 172 rooms on nine sections, the label sits a median of 0.24 room-equivalent radii from the geometric centroid of its region, with a mean of 0.27 and a worst case of 0.98, and 92% of labels fall inside the room’s inscribed circle. Architectural drafting practice places labels centrally, so seeding main nodes at labels is empirically sound.

10.3 Connectivity under Door-Detector Degradation

Door detection is the dominant failure source, so we quantify how detector errors propagate to connectivity. The experiment deletes doors from the pre-pruning graph, where the full corridor and room mesh is intact, and measures pairwise room completeness and exit reachability with 20 Monte-Carlo draws per level. Deletions apply to the detector output because no door-level ground truth exists for these drawings, so the curves measure sensitivity relative to the current detector. Figure 3 shows the curves for random deletion and for deletion restricted to room-to-corridor and corridor-to-corridor doors.

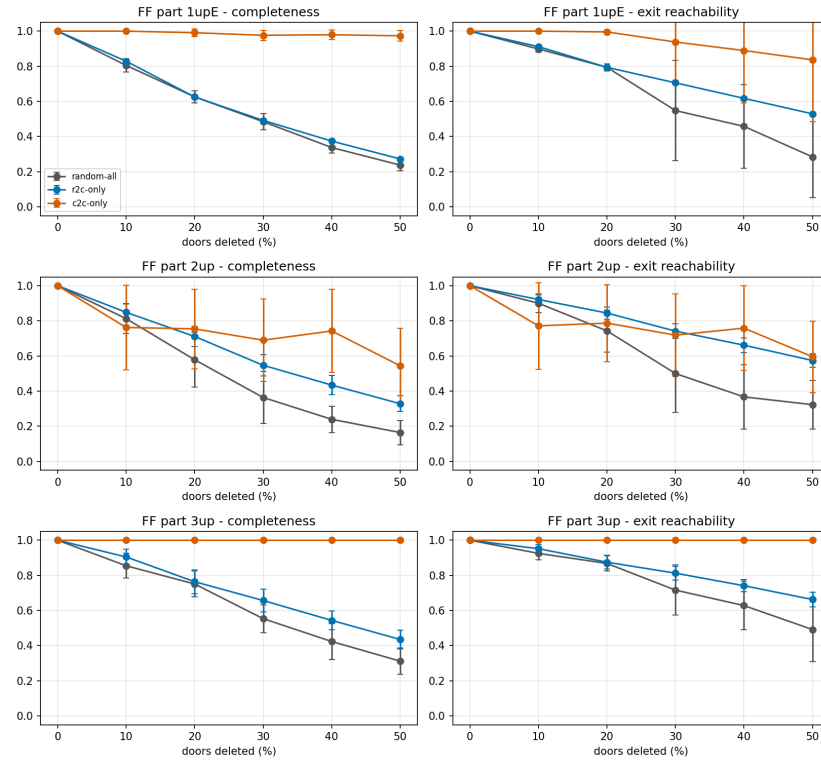


Fig. 3. Connectivity under door deletion for three floor sections. Random deletion and room-to-corridor deletion degrade completeness steadily because each room is single-homed through one access door. Corridor-to-corridor deletion barely moves two of the floors and bites on the third, which motivates the per-door analysis.

Per-door analysis sharpens the picture. Deleting each corridor-to-corridor door alone shows that most such doors are redundant because the corridor mesh reconnects around them. The exceptions are bridge doors whose removal disconnects entire sections. On first-floor section B a single such door costs 0.51 of completeness and 0.54 of exit reachability by itself. Bridge doors coincide with the cut vertices of the graph, so they are mechanically identifiable, which turns the dominant failure mode into a targeted verification problem. Section 11 outlines the resulting repair direction.

10.4 Graph Quality, Ablations, and Pruning

Graph quality is measured with an exhaustive metric layer. Completeness is the fraction of unordered main-room pairs connected by a path, exit reachability is the fraction of main rooms that reach at least one exit door, and both are computed from the full component structure rather than from samples. Most processed sections are fully connected. Three sections, among them section D of Table 2, are fragmented as built with completeness between 0.51 and 0.76, which quantifies the corridor-door failure mode on real output.

Funneling Ablation. Three intra-room connectivity strategies are compared on the same floors, namely dense wiring from every family node to every family door, funneling, and a single node per room. Table 3 reports the outcome. All variants achieve complete family-to-door connectivity, and they differ in cost and route quality. Funneling retains 167 intra-room edges against 1,869 for dense wiring on section A, at a route stretch of 7.2% over the dense reference, while the single-node variant pays 13.5% and forfeits intra-room spatial fidelity. A second floor yields exactly one intra-room edge per subnode, which matches Proposition 7.1.

Table 3. Funneling ablation on first-floor section A. Route lengths are subnode-to-nearest-exit means, and stretch is relative to dense wiring.

Variant	Nodes	Edges	Completeness	Route stretch	Query (ms)
Dense wiring	878	1,273	1.000	1.000	0.44
Funneling	878	1,118	1.000	1.072	0.40
One node per room	680	918	1.000	1.135	0.35

Pruning Ablation. Each pruning keep-term is load-bearing. Re-deriving the keep-set with individual terms disabled shows that dropping the transition term silently removes every transition from the pruned graph, and dropping the exit term removes every exit door. The full pruning pass removes 59% of nodes and 82% of edges at a measured route inflation of exactly zero, because kept paths are shortest paths by construction, while mean single-pair query time improves from 2.10 to 0.50 milliseconds. Figure 4 illustrates the reduction. This conservative keep-set is intentionally redundant, and the measured outcome justifies it, since connectivity and route quality are fully preserved at a fraction of the graph size.

10.5 Corridor Sampling Density

Corridor node density buys a safety margin rather than reachability. Rerunning the full pipeline at grid spacings from 20 to 80 pixels shows that a 60-pixel grid keeps the graph fully connected with room-to-exit routes within 4.6% of the 20-pixel default at 57% fewer nodes. At 80 pixels the corridor mesh fragments, completeness collapses to 0.49, and routes err by 163%. The failure mode is a cliff rather than a slope, so the dense default guards against narrow corridors at a known cost. Figure 5 plots the trade-off.

10.6 Route Fidelity against Free-Space Geodesics

Route quality is validated against an objective walkable reference. The reference is the free-space geodesic, the exact shortest path through the drawing’s non-wall pixels with doorways opened at detected door positions. Straight-line distance is a poor reference because walls force real detours, and the measured ratio of geodesic to Euclidean distance runs between 1.35 and 1.67 on average with a maximum of 3.3. We report the geodesic reference in two variants that

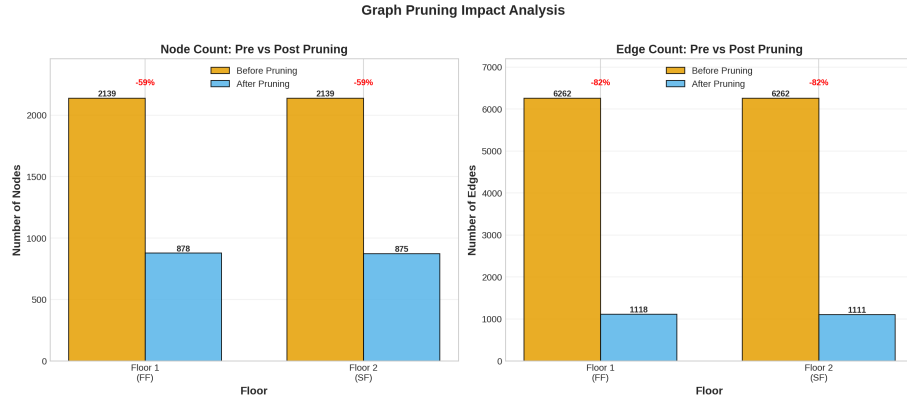


Fig. 4. Pre-pruning and post-pruning graph sizes on the two-floor merge.

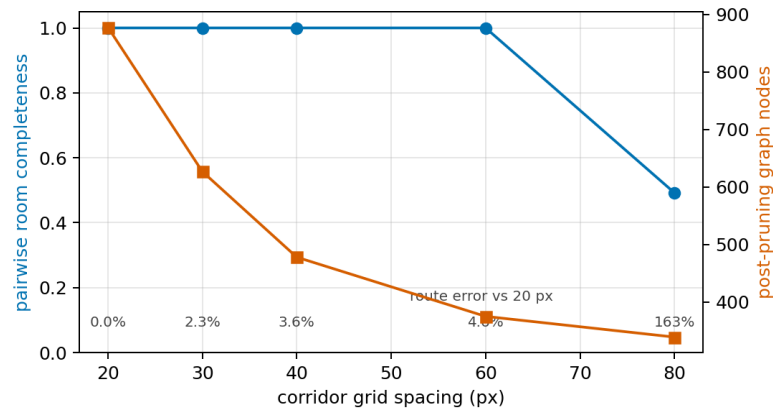


Fig. 5. Corridor density trade-off on first-floor section A. Completeness holds from 20 to 60 pixels and collapses at 80, while graph size grows steeply toward dense spacing. Annotations give the mean route error against the 20-pixel reference.

bracket the truth, a permissive bound over all free space and a conservative bound that excludes the semantic outside class.

Graph routes track the geodesics closely on enclosed sections. On first-floor section A the stretch of graph routes over geodesics has a mean of 1.047 and a ninetieth percentile of 1.093 under both bounds. On the more open section B the conservative bound gives a mean stretch of 1.24 while the permissive bound gives 2.30, and the gap between the bounds quantifies sparse outdoor routing as a concrete limitation. Figure 6 shows a verification overlay.

10.7 Baseline Comparison

A structure-only baseline isolates the value of the typed construction. The baseline skeletonizes free space into a medial-axis routing backbone and receives our own room and exit detections, so the comparison targets routing structure alone. Table 4 reports the outcome on the same room pairs and the same geodesic reference. The skeleton needs between 3.7 and 12 times more nodes, fails completeness because it fragments at doorway gaps, produces worse route

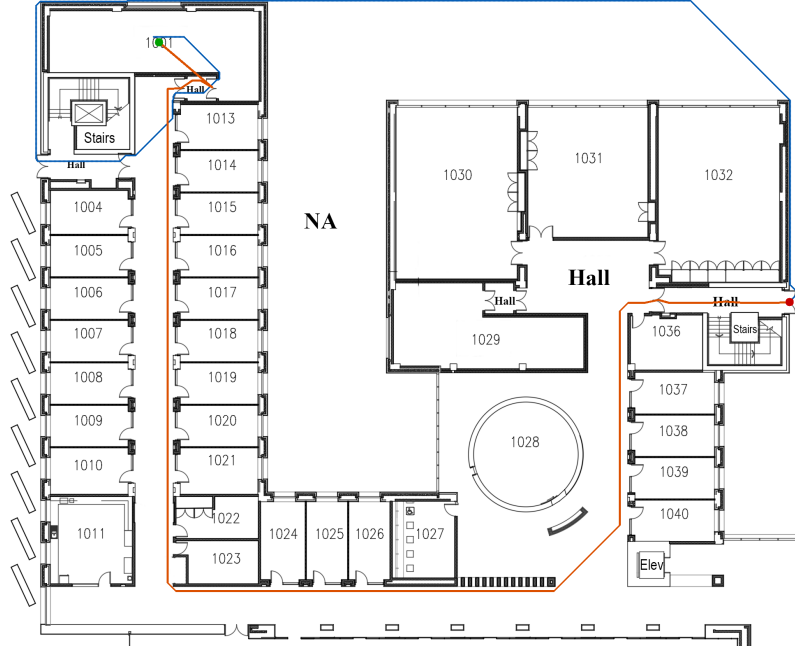


Fig. 6. Verification overlay for one room-to-exit pair. The dotted trace is the free-space geodesic and the solid polyline is the graph route.

stretch, and answers queries between 37 and 94 times slower. It also expresses none of the typed downstream tasks, since it has no rooms, doors, or transitions as objects.

Table 4. Medial-axis baseline versus TESSERACT++ on two sections. Stretch is against the interior geodesic reference.

Section	Graph	Nodes	Edges	Completeness	Stretch	Query (ms)
A	Skeleton	3,248	2,029	0.829	1.233	14.2
A	TESSERACT++	878	1,118	1.000	1.047	0.38
B	Skeleton	8,598	6,959	0.782	1.381	34.0
B	TESSERACT++	712	928	1.000	1.231	0.36

10.8 Multi-Floor Evaluation

Matching Robustness. The automatic matcher is evaluated under controlled perturbations of a real floor. A synthetic counterpart floor is generated by a known similarity transform plus five pixels of position jitter, so correctness is exact by construction. Matching accuracy is 1.000 across every sweep, namely translations to 400 pixels, rotations to 30 degrees, scale changes from 0.7 to 1.4, twin-shaft separations down to 25 pixels, and shaft-deletion tests in which the deleted shaft's partner must remain unmatched. A greedy nearest-neighbor baseline degrades to 0.75 accuracy on close twins. Median registration recovery error is 0.5 pixels under translation, 4.4 under rotation, and 4.9 under scale. Figure 7 shows the accuracy panels and Figure 8 the recovery envelope.

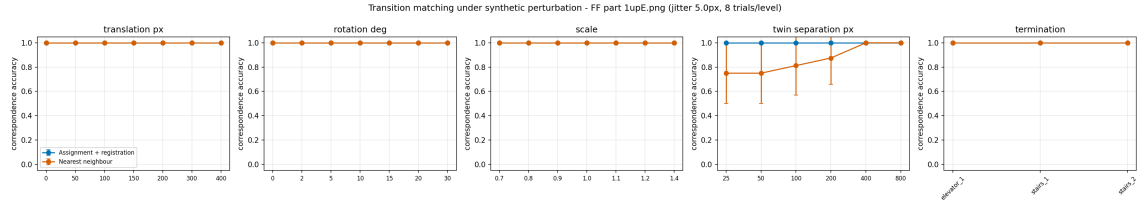


Fig. 7. Matching accuracy under synthetic perturbation. The assignment matcher holds 1.000 throughout while the nearest-neighbor baseline collapses as twin shafts approach.

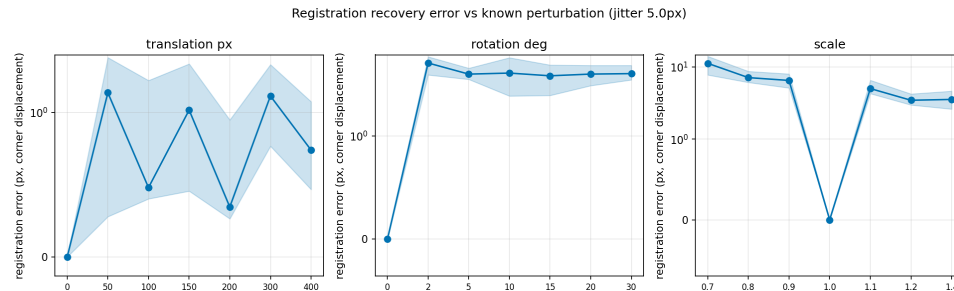


Fig. 8. Registration recovery error against known perturbations, shown as median with interquartile band.

Cross-Floor Case Study. A genuinely different floor pair exercises the conservative behavior. Matching the ground-floor section against the first-floor section, two drawings with different crops and extents, the door-width prior fixes the relative scale at 1.036, registration selects a single supported correspondence at a confidence of 0.83, and every remaining shaft on both floors is rejected rather than force-paired. The system flags the weak wall agreement and the single supporting pair in the mapping provenance, which is exactly the reviewable outcome intended for partially overlapping sections.

Merged Graphs. The merged two-floor graphs are fully connected, and every room reaches every other room and at least one exit. With modeled vertical travel weights the measured step-free detour ratio corrects from 2.02 to 1.79, which shows that nominal unit weights had inflated cross-floor route costs.

10.9 Downstream Task Validation

Two navigation tasks validate the representation directly. Step-free routing forbids stairs and is expressible only because transitions are typed. On the two-floor merge with full transition correspondence, all 200 sampled cross-floor room pairs remain reachable without stairs, at a mean detour of 1.79 and a ninetieth percentile of 3.32 relative to unrestricted routing. On the ground-to-first-floor pair, where only a stairs correspondence is established, all 102 pairs are reachable normally and none is reachable step-free, which surfaces an accessibility gap that untyped graphs cannot express. Egress analysis computes each room's distance to its nearest exit by multi-source Dijkstra, and every room on the evaluation floors has an egress path with a median distance of 901 pixels and a ninetieth percentile of 1,503.

10.10 Timing

Table 5 reports the per-stage breakdown on first-floor section A.

Table 5. Single-floor processing time breakdown for a 1946×1574 drawing.

Stage	Time (s)
Text detection	3.0
Text interpretation	2.5
Flood-fill segmentation	9.0
Door detection	21.2
Door classification	2.9
Node updates and entry doors	4.8
Edge creation	0.1
Pruning	2.0
Total	52.0

10.11 Qualitative Results

Figure 9 visualizes a merged two-floor graph with its inter-floor connections.

Registration and Matching. Automatic matching depends on registration evidence. Partially overlapping sections yield weak wall agreement, mirrored drawings are detected and flagged rather than corrected silently, and floors that share fewer than two true correspondences rely on the door-width scale prior. The system reports these conditions as explicit warnings, and a hand-written mapping remains the override for cases the evidence cannot settle.

Outdoor Routing. The gap between the permissive and conservative fidelity bounds on open sections shows that outdoor routing is sparser than indoor routing. Denser outdoor sampling would narrow this gap at a known node cost.

Computational Requirements. Neural network inference dominates processing time. GPU acceleration would reduce latency for deployment scenarios.

Ground Truth Availability. Quantitative evaluation remains limited by the absence of standardized floorplan-to-graph benchmarks. The free-space geodesic reference reduces this dependency for route quality, and we encourage community development of annotated datasets for full-graph comparison.

12 CONCLUSION

We presented TESSERACT++, an end-to-end system that extracts navigable graph representations from annotated building floorplans. The pipeline integrates text detection, semantic interpretation, spatial segmentation, door detection, and typed graph construction, and it extends to multi-floor buildings through automatic transition matching and validated merging.

The revision establishes four main results. Room-family funneling carries a proven guarantee of complete intra-room connectivity at linear intra-room edge cost, and an ablation places it at the size-quality sweet spot. Vertical correspondences are inferred automatically through registration and assignment with rejection, with perfect accuracy under controlled perturbation and conservative, warning-guarded behavior on partially overlapping real sections. Graph routes stay within 4.7% of exact free-space geodesics on the primary evaluation floor, and pruning removes 59% of nodes and 82% of edges at zero measured route cost. A structure-only medial-axis baseline confirms that the typed construction wins on size, completeness, route quality, and query time simultaneously.

Future work includes image-guided repair of bridge doors, extension to non-annotated floorplans through learned segmentation, directional edge attributes, and standardized benchmarks for floorplan-to-graph evaluation.

ACKNOWLEDGMENTS

REFERENCES

- [1] Imad Afyouni, Cyril Ray, and Christophe Claramunt. 2010. A Fine-Grained Context-Dependent Model for Indoor Spaces. (2010), 33–38.
- [2] Imad Afyouni, Cyril Ray, and Christophe Claramunt. 2012. Spatial Models for Context-Aware Indoor Navigation Systems: A Survey. *Journal of Spatial Information Science* 4 (2012), 85–123.
- [3] Yaqoob Ansari, Eduardo Feo Flushing, and Khaled A. Harras. 2025. Tesseract: Unfolding Navigable Graph Representations from Low-Semantic Floor Plans. In *Proceedings of [Conference Name]*. [UPDATE: Add actual conference venue and details].
- [4] Youngmin Baek, Bado Lee, Dongyoon Han, Sangdoo Yun, and Hwalsuk Lee. 2019. Character Region Awareness for Text Detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 9365–9374.
- [5] Paramvir Bahl and Venkata N. Padmanabhan. 2000. RADAR: An In-Building RF-Based User Location and Tracking System. In *Proceedings of IEEE INFOCOM*, Vol. 2. 775–784.
- [6] Gunilla Borgefors. 1986. Distance Transformations in Digital Images. *Computer Vision, Graphics, and Image Processing* 34, 3 (1986), 344–371.
- [7] buildingSMART International. 2023. Industry Foundation Classes (IFC) Specification. <https://www.buildingsmart.org/standards/bsi-standards/industry-foundation-classes/>. Accessed: 2024-01-01.
- [8] Cesar Cadena, Luca Carlone, Henry Carrillo, Yasir Latif, Davide Scaramuzza, José Neira, Ian Reid, and John J. Leonard. 2016. Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age. *IEEE Transactions on Robotics* 32, 6 (2016), 1309–1332.

- [9] Edsger W. Dijkstra. 1959. A Note on Two Problems in Connexion with Graphs. *Numer. Math.* 1, 1 (1959), 269–271.
- [10] Samuel Dodge, Jiu Xu, and Bjorn Stenger. 2017. Parsing Floor Plan Images. (2017), 358–361.
- [11] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. 2006. Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks. (2006), 369–376.
- [12] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. 2008. Exploring Network Structure, Dynamics, and Function using NetworkX. In *Proceedings of the 7th Python in Science Conference*. 11–15.
- [13] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. 1968. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics* 4, 2 (1968), 100–107.
- [14] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 770–778.
- [15] Ahti Kalervo, Juha Ylioinas, Markus Häikiö, Antti Karhu, and Juho Kannala. 2019. CubiCasa5K: A Dataset and an Improved Multi-Task Model for Floorplan Image Analysis. In *Scandinavian Conference on Image Analysis*. Springer, 28–40.
- [16] Hae-Kyong Kang and Ki-Joune Li. 2017. A Standard Indoor Spatial Data Model—OGC IndoorGML and Implementation Approaches. *ISPRS International Journal of Geo-Information* 6, 4 (2017), 116.
- [17] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. 2017. Feature Pyramid Networks for Object Detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2117–2125.
- [18] Chen Liu, Jiaye Wu, and Yasutaka Furukawa. 2018. FloorNet: A Unified Framework for Floorplan Reconstruction from 3D Scans. In *Proceedings of the European Conference on Computer Vision (ECCV)*. 201–217.
- [19] Chen Liu, Jiajun Wu, Pushmeet Kohli, and Yasutaka Furukawa. 2017. Raster-to-Vector: Revisiting Floorplan Transformation. (2017), 2195–2203.
- [20] Martin Raubal and Max J. Egenhofer. 1998. Comparing the Complexity of Wayfinding Tasks in Built Environments. *Environment and Planning B: Planning and Design* 25, 6 (1998), 895–913.
- [21] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. 2015. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 28.
- [22] Baoguang Shi, Xiang Bai, and Cong Yao. 2017. An End-to-End Trainable Neural Network for Image-Based Sequence Recognition and Its Application to Scene Text Recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 39, 11, 2298–2304.
- [23] Rebekka Volk, Julian Stengel, and Frank Schultmann. 2014. Building Information Modeling (BIM) for Existing Buildings—Literature Review and Future Needs. *Automation in Construction* 38 (2014), 109–127.
- [24] Xinyu Zhou, Cong Yao, He Wen, Yuzhi Wang, Shuchang Zhou, Weiran He, and Jiajun Liang. 2017. EAST: An Efficient and Accurate Scene Text Detector. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 5551–5560.

A IMPLEMENTATION DETAILS

A.1 Pipeline Parameters

The wall threshold is 240 on the grayscale intensity. Room subnodes use hexagonal spacing of 60 pixels with a 10-pixel wall padding and 4-pixel jitter. Corridor sampling defaults to a 20-pixel grid, configurable per run. Outdoor sampling uses a 40-pixel grid.

A.2 Matching Parameters

Wall masks for registration are downsampled to 400 pixels on the long side, and connected components below 30 square pixels are removed as glyphs. Alignment counts a pair within 2% of the image diagonal. Plausibility gates accept scales between 0.4 and 2.5 and rotations up to 60 degrees, and chamfer scoring requires at least 25% mutual coverage. The rejection price is 0.10, and the cost weights are 1.0 for position, 1.0 for kind mismatch, and 0.05 for graph context. Inter-floor weights use a 0.9-meter door leaf, a 4-meter story height, and travel factors of 2.2 for stairs and 1.0 for elevators.

B ADDITIONAL VISUALIZATIONS

Figures 10 and 11 reproduce the evaluation drawings of Figure 2 at full page width so that labels, door symbols, and wall conventions are legible.

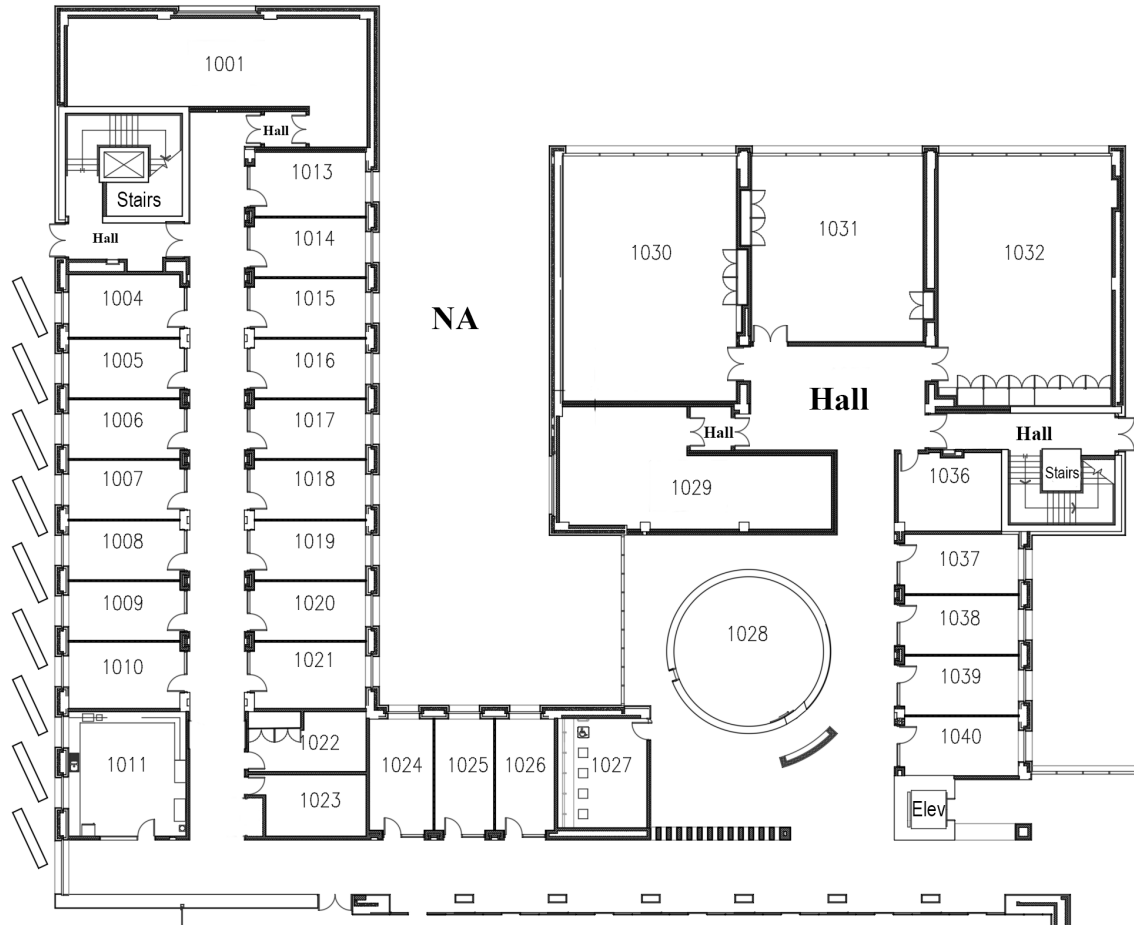


Fig. 10. First-floor section A at full width.

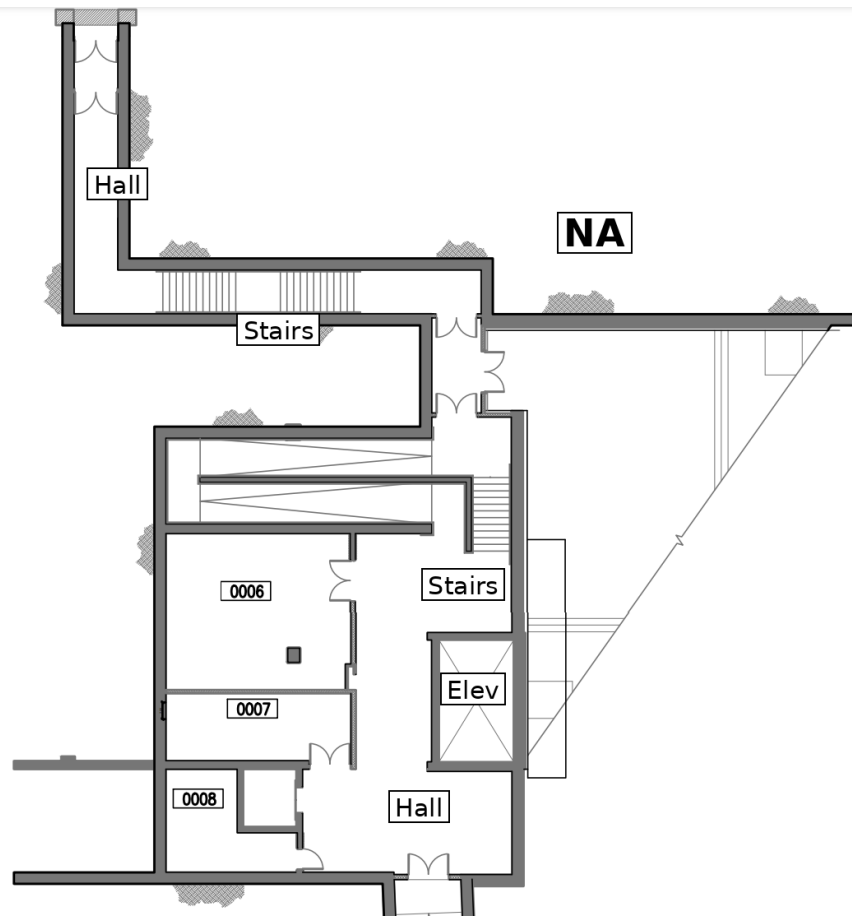


Fig. 11. Ground-floor section at full width.

C JSON SCHEMA

Exported graphs contain a node list and an edge list. Nodes carry `id`, `type`, `position`, `floor`, and `pixels`, with room nodes adding `is_subnode`, `parent_room_id`, area statistics, and anchor-door references, and door nodes adding sub-type and connectivity references. Edges carry `source`, `target`, `weight`, `distance`, and `edge_type`. Merged graphs add `original_id` and `source_image` per node.